

List Operations

TRAVERSAL, INSERTION, & DELETION

DSA Mastery Series: Part 18

1. Traversing the Chain

Traversal is the foundational operation for all other list actions. Since linked list nodes are scattered in memory, we must follow the "next" pointer breadcrumbs one by one starting from the **Head**.



```
# Python Traversal Implementation
def traverse(head):
    currentNode = head
    while currentNode:
        print(currentNode.data)
        currentNode = currentNode.next
```

Time Complexity: $O(n)$

2. Node Deletion

Deleting a node requires surgery on the pointers. You must bridge the **previous** node to the node **following** the one you want to delete.

The Logic:

In a Singly Linked List, you must traverse until you find the node before the one to be deleted, because you cannot move backwards.

```
def deleteNode(head, targetNode):
    if head == targetNode:
        return head.next

    curr = head
    while curr.next and curr.next != targetNode:
        curr = curr.next

    if curr.next:
        # Bridging: Skip the middle node
        curr.next = curr.next.next
    return head
```

3. Node Insertion

Insertion is the process of creating a new node and "sewing" it into the existing list. Order happens in two sub-steps to avoid losing the rest of the list.

```
def insert(head, newNode, position):
    if position == 1:
        newNode.next = head
        return newNode

    curr = head
    for _ in range(position - 2):
        curr = curr.next

    # Step 1: Link new node to the tail of the list
    newNode.next = curr.next
    # Step 2: Link previous part to the new node
    curr.next = newNode
    return head
```

4. Operations vs. Arrays

Binary Search Problem

Binary Search is **impossible** on Linked Lists because it relies on "middle" access ($O(1)$ in arrays), whereas finding the middle of a list requires $O(n)$ traversal.

Sorting Limitations

While you can use **Selection Sort** on lists, you cannot use algorithms that require random index access, such as **Radix Sort** or **Counting Sort**.

Operation	Array	Linked List
Access Index	$O(1)$	$O(n)$
Insert (Start)	$O(n)$	$O(1)$
Find Lowest	$O(n)$	$O(n)$

5. Knowledge Check

Code Challenge:

```
currentNode = head
while currentNode:
    print(currentNode.data)
    currentNode = currentNode._____
```

Answer: .next

© 2026 codeHive Academy • Essential List Manipulations